This would also have been a valid way of implementing the stack. So, if you exclude the `wait_and_pop()` functions, the analysis you did for the stack applies just as well here.

The new `wait_and_pop()` functions are a solution to the problem of waiting for a queue entry that you saw with the stack; rather than continuously calling `empty()`, the waiting thread can just call `wait_and_pop()` and the data structure will handle the waiting with a condition variable. The call to `data_cond.wait()` won't return until the underlying queue has at least one element, so you don't have to worry about the possibility of an empty queue at this point in the code, and the data is still protected with the lock on the mutex. These functions don't therefore add any new race conditions or possibilities for deadlock, and the invariants will be upheld.

There's a slight twist with regard to exception safety in that if more than one thread is waiting when an entry is pushed onto the queue, only one thread will be woken by the call to `data_cond.notify_one()`. However, if that thread then throws an exception in `wait_and_pop()`, such as when the new `std::shared_ptr<>` is constructed ❹, none of the other threads will be woken. If this isn't acceptable, the call is readily replaced with `data_cond.notify_all()`, which will wake all the threads but at the cost of most of them then going back to sleep when they find that the queue is empty after all. A second alternative is to have `wait_and_pop()` call `notify_one()` if an exception is thrown, so that another thread can attempt to retrieve the stored value. A third alternative is to move the `std::shared_ptr<>` initialization to the `push()` call and store `std::shared_ptr<>` instances rather than direct data values. Copying the `std::shared_ptr<>` out of the internal `std::queue<>` then can't throw an exception, so `wait_and_pop()` is safe again. The following listing shows the queue implementation revised with this in mind.

Listing 6.3. A thread-safe queue holding `std::shared_ptr<>` instances